

# REST API Notes

## 1. What is a REST API?

**REST API (Representational State Transfer API)** is a way for a **client and server to communicate over HTTP**.

Data is commonly exchanged in **JSON format**.

### Basic flow

Client → HTTP Request → Server

Client ← HTTP Response ← Server

A REST API:

- Uses HTTP methods such as **GET, POST, PUT, PATCH, DELETE**
- Uses **URLs/endpoints** to identify resources
- Usually exchanges data in **JSON**
- Maps HTTP methods to **CRUD operations**

### REST vs HTTP

- **REST** → Architectural style that defines how an API should be designed.
- **HTTP** → Protocol used to transfer requests and responses.

They work together, but **REST and HTTP are not the same thing**.

## 2. HTTP Methods in REST API

| HTTP Method   | CRUD Operation | Purpose                        |
|---------------|----------------|--------------------------------|
| <b>GET</b>    | Read           | Retrieve data                  |
| <b>POST</b>   | Create         | Create new data                |
| <b>PUT</b>    | Update         | Replace/update entire resource |
| <b>PATCH</b>  | Update         | Partially update resource      |
| <b>DELETE</b> | Delete         | Delete data                    |

## 3. GET

**GET** is used to retrieve a resource.

Example:

GET /users/123

This requests the user whose ID is 123.

Common responses:

- 200 OK → Request successful
- 404 Not Found → Resource doesn't exist
- 400 Bad Request → Invalid request

## Example

GET /users

Could return:

```
[
  {
    "id": 1,
    "name": "Anne"
  },
  {
    "id": 2,
    "name": "John"
  }
]
```

# 4. POST

**POST** is used to create a new resource.

Example:

POST /users

Request body:

```
{
  "name": "Anne",
  "email": "anne@example.com"
}
```

The server creates a new user.

Successful creation commonly returns:

201 Created

## Important

**POST is not idempotent.**

Sending the same POST request multiple times can create multiple resources.

## 5. PUT

**PUT** is generally used to **replace an entire resource**.

Example:

PUT /users/123

Request body:

```
{  
  "name": "Anne",  
  "email": "anne@example.com"  
}
```

The complete user information is sent.

### PUT characteristics

- Usually sends the **complete resource**
- Replaces the existing resource
- **Idempotent**

## 6. PATCH

**PATCH** is used for a **partial update**.

Example:

PATCH /users/123

Request:

```
{  
  "email": "newemail@example.com"  
}
```

Only the email is changed.

Other user information remains unchanged.

## 7. PUT vs PATCH

| PUT                            | PATCH                         |
|--------------------------------|-------------------------------|
| Replaces the entire resource   | Updates only specified fields |
| Usually sends complete data    | Sends only changed data       |
| Idempotent                     | Not necessarily idempotent    |
| Used for complete replacement  | Used for partial updates      |
| Example: Update entire profile | Example: Change only email    |

**PUT → Replace**

**PATCH → Modify a part**

## 8. DELETE

**DELETE** is used to delete a resource.

Example:

DELETE /users/123

This deletes the user with ID 123.

A successful response can be:

200 OK

or commonly:

204 No Content

## 9. Idempotence

An HTTP method is **idempotent** if sending the same request multiple times produces the **same final server state** as sending it once.

### Example

Suppose user 123 has:

```
{  
  "name": "Anne"
```

```
}
```

Sending:

PUT /users/123

with:

```
{  
  "name": "John"  
}
```

once or multiple times leaves the user as:

```
{  
  "name": "John"  
}
```

So PUT is **idempotent**.

### Simple example

PUT → Set value to 10

PUT → Set value to 10

PUT → Set value to 10

Final value is still:

10

Whereas:

POST → Create user

POST → Create user

could create two users.

## 10. Building REST API Using Node.js + Express

You created a Node.js project using:

```
mkdir node-app  
cd node-app
```

Then initialized it:

```
npm init -y
```

This creates:

package.json

Then installed Express:

```
npm install express
```

## 11. Express REST API

Your server . js contains four main API endpoints:

```
GET    /users
POST   /users
PUT    /users/:id
DELETE /users/:id
```

### Important Express code

```
const express = require('express');
const app = express();
const port = 3000;
```

```
app.use(express.json());
```

### **express.json()**

This middleware allows Express to read **JSON data sent in the request body**.

For example:

```
{
  "name": "Priyanka",
  "email": "priyanka@gmail.com"
}
```

The data can then be accessed using:

```
req.body
```

## 12. GET /users

Your code:

```
app.get('/users', (req, res) => {
  res.json({ message: 'Returning list of users' });
});
```

When the client sends:

GET http://localhost:3000/users

the server returns:

```
{  
  "message": "Returning list of users"  
}
```

## 13. POST /users

Your code:

```
app.post('/users', (req, res) => {  
  const newUser = req.body;  
  res.json({ message: 'User created', user: newUser });  
});
```

The client sends:

POST http://localhost:3000/users

with:

```
{  
  "name": "Archana",  
  "email": "archana@gmail.com"  
}
```

req.body contains:

```
{  
  "name": "Archana",  
  "email": "archana@gmail.com"  
}
```

## 14. PUT /users/:id

Your code:

```
app.put('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  const updatedUser = req.body;  
  
  res.json({  
    message: `User with ID ${userId} updated`,  
    updatedUser  
  });  
});
```

Here:

/users/:id

is a **dynamic route**.

For example:

PUT /users/1

Then:

req.params.id

contains:

1

And:

req.body

contains the updated user data.

## 15. DELETE /users/:id

Your code:

```
app.delete('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  
  res.json({  
    message: `User with ID ${userId} deleted`  
  });  
});
```

Request:

DELETE /users/1

Then:

req.params.id

is:

1

Response:

```
{  
  "message": "User with ID 1 deleted"  
}
```



## 16. Starting the Server

Run:

```
node server.js
```

You should see:

Server is running on `http://localhost:3000`

Your server is now listening on **port 3000**.

## 17. Testing REST API Using Postman

You can test your endpoints using **Postman**.

### GET

GET `http://localhost:3000/users`

### POST

POST `http://localhost:3000/users`

Body → **raw** → **JSON**

```
{
  "name": "Archana",
  "email": "archana@gmail.com"
}
```

### PUT

PUT `http://localhost:3000/users/1`

Body:

```
{
  "name": "Archana",
  "email": "archana@gmail.com"
}
```

### DELETE

DELETE `http://localhost:3000/users/1`

## 18. Why Cannot GET / Appeared

You previously opened:

`http://localhost:3000/`

But your server only defines:

`/users`

There is no:

`app.get('/', ...)`

Therefore Express displayed:

Cannot GET /

Your API is working correctly.

You should instead test:

`http://localhost:3000/users`

## 19. Features of REST APIs

### 1. Statelessness

Every request contains the information needed to process it.

The server does not depend on storing the client's session state between requests.

### 2. Client-Server Architecture

Client and server are separated.

Frontend → REST API → Backend

This provides separation of concerns and improves scalability.

### 3. Cacheable

Responses can be cached when appropriate.

Caching can:

- Improve performance
- Reduce server load
- Reduce repeated requests

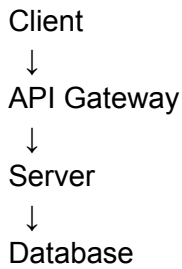
### 4. Uniform Interface

REST follows consistent conventions involving:

- URLs
- HTTP methods
- Representations
- Status codes

## 5. Layered System

A REST system can contain multiple layers such as:



The client does not necessarily need to know about all internal layers.

# 20. Real-World Uses of REST APIs

## Social Media

Used for:

- Login
- Posts
- Likes
- Comments
- Sharing

## E-Commerce

Used for:

- Products
- Orders
- Payments
- Customers
- Inventory

## Geolocation

Used for:

- GPS tracking
- Location updates
- Nearby places

## Weather Applications

A frontend can call a weather API to retrieve:

Temperature  
Humidity  
Wind  
Forecast

## 21. Limitations of REST APIs

- Requests can become large because required information must be included.
- REST uses a request-response model and isn't ideal for some real-time applications.
- Clients can sometimes receive **more or less data than they actually need**.
- REST provides architectural constraints rather than one rigid implementation standard.
- API versioning and backward compatibility can become challenging as an API grows.